‹packt›

The Spring Pocket Guide

Josh Long

The Spring Pocket Guide

Portfolio Director: Ashwin Nair
Relationship Lead: Aaron Lazar
Project Manager: Ruvika Rao
Content Engineer: Nithya Sadanandan
Technical Editor: Rohit Singh
Copy Editor: Safis Editing
Indexer: Rekha Nair
Proofreader: Nithya Sadanandan
Production Designer: Deepak Chavan
Production reference: 1130326

Published by Packt Publishing Ltd.
Grosvenor House
11 St Paul's Square
Birmingham
B3 1RB, UK

ISBN 978-1-80760-179-9
www.packtpub.com

About the author

Josh Long is a Spring Developer Advocate at Broadcom and has served as the first Spring Developer Advocate since 2010. An engineer, open-source contributor, and educator, he has helped tens of millions of developers reach production with cloud-native technologies such as Spring Boot, Spring AI, Cloud Foundry, and Kubernetes through his articles, videos, books, blogs, courses, and conference talks. Josh contributes to numerous open-source projects, including Spring Boot, Spring AI, Spring Cloud, Spring Integration, Spring Batch, Spring Modulith, JobRunr, Flowable, Vaadin, and MyBatis. He also runs the Coffee Software YouTube channel, and is a Java Champion, Kotlin GDE (alumnus), Microsoft MVP, and lifelong student of new and emerging technologies.

Preface

Spring has long been the backbone of modern Java development, empowering teams to build secure, scalable, and production-ready applications with confidence. From startups to global enterprises, developers rely on Spring to move quickly without sacrificing architectural integrity. By combining powerful abstractions with thoughtful defaults, Spring allows you to focus on business value while the framework handles the heavy lifting.

In this nano book, you will take a fast-paced journey through the essentials of modern Spring web development. Starting with project setup using Spring Boot and the Initializr, you will explore how to build HTTP APIs, model data with Spring Data JDBC, and expose services using REST, HATEOAS, GraphQL, and gRPC. Along the way, you will integrate PostgreSQL, consume external APIs, and see how Spring's component model keeps applications clean, modular, and maintainable.

You will also dive into production-critical concerns such as security with Spring Security and OAuth 2, API gateways, configuration management, scalability with virtual threads, and performance optimization using GraalVM native images. Finally, you will explore how Spring AI, vector databases, tool calling, and the **Model Context Protocol** (**MCP**) bring intelligent, AI-powered capabilities into modern applications.

By the end of this guide, you will understand not just how to build a Spring application, but also how to design systems that are secure, scalable, extensible, and ready for real-world production. Whether you are modernizing an existing system or starting fresh, Spring provides the foundation, and this book will help you use it with clarity and confidence.

Who this book is for

This pocket guide will act as a quick starter for anyone who wants to familiarize themselves with the Spring ecosystem for building modern web apps. Whether you're a developer, a tech lead, or a business owner who's contemplating using Spring for your projects, you'll get a clear overview of how Spring can help solve your business problem.

Download the example code files

You can download the example code files for this book from GitHub at `https://github.com/PacktPublishing/The-Spring-Pocket-Guide`. If there's an update to the code, it will be updated in the GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at `https://github.com/PacktPublishing/`. Check them out!

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book or have any general feedback, please email us at `customercare@packt.com` and mention the book's title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported it to us. Please visit `http://www.packt.com/submit-errata`, click **Submit Errata**, and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you provided us with the location address or website name. Please contact us at `copyright@packt.com` with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit http://authors.packt.com/.

Important!

Hands-On Workshop

Join us for a hands-on workshop on **31st March**, **2026** and learn how to take advantage of Spring Boot, Spring Cloud, and other tools within the Spring ecosystem to build robust microservices-based applications.

Special Discount: Use the code **SPG35** to get your passes for **35%** off.

Table of Contents

The Spring Pocket Guide

So, you've decided you want to win on the web, have you?

The web is amazing! And I mean the real web, not those other webs we keep hearing about. No, the web I'm talking about isn't dark, deep, Web3, AOL, or spidery. No, no, I mean the real web. The one that uses good old-fashioned HTTP and a few adjacent protocols. Look, Ma, no blockchain required!

Oh, sure, it might be a little old, a little quaint… but this web? It'll surprise you. It's still got a few amazing tricks left, and I bet you're going to love it. It's going to be huge.

It already is huge. How many apps are there on the iPhone App Store? As of my last check, at the end of 2024, it was hovering around 1.8 to 2 million. What about the Android store? 2.5 to 3 million, you say? Nice! So, that's around five million apps. And that's not even counting all the other app store experiences out there, from the Metaverse (with its five users) to Apple Vision Pro (with its four users), Steam, and whatever else.

Do you know how many HTTP endpoints are out there? By that, I mean pages and APIs on the web.

No? Yeah. Me either!

But I'll bet it's north of a few hundred billion, or even a trillion. It's enormous! Due, of course, in no small part to how many of those apps mentioned previously rely on web services built on HTTP! But also, just because the web is a natural place to build! The web's awesome, and Spring is the key that opens up that door.

Every large unicorn I can think of (yes, even *them*!) except Google (because they built scalable web technology before Spring existed) or Facebook (because they're too proud to admit PHP was an awful mistake and instead have spent decades doubling down, trying to make PHP a half-implemented, bug-riddled implementation of Java 1.5) use Spring. You know how I know? They're hiring for Spring developers, or they've given tech talks or keynotes asserting as much. Spring's the thing if you want to get to production quickly and prosper once you're there.

In this text, we'll learn how to use Spring and then look at how others are doing so to their benefit.

Bootcamp

You'll need a few things installed before beginning the adventure to production.

Make sure you've got `SDKMAN.io` installed. Use that to ensure you've got **Java 24 or later** installed. If you can, make sure you're using **GraalVM for Java {java-version}**. Here's how I installed it:

```
sdk install java 24-graalce
sdk default java 24-graalce
```

Ensure that Docker Compose is installed.

These are the tools of the modern web architect.

Your first project

Let's build a service called `adoptions` to support adopting dogs from a fictitious dog adoption agency called *Pooch Palace*. We're going to build stuff. *A lot of stuff*. But we'll always start at the Spring Initializr. Choose **Maven** for **Project** and **Java 24** (or later) under **Language**. At the time of this writing, Spring Boot 3.5.x was the mainline. Very likely, Spring Boot 4 or later will be out by the time you read this. For now, if Spring Boot 4 is not GA, prefer Spring Boot 3.5.x.

Here's the recipe for `adoptions`: **Spring Web**, **Spring Data JDBC**, **Spring for GraphQL**, **Spring gRPC**, **GraalVM Native Support**, **Spring Boot DevTools**, **Spring HATEOAS**, **Pgvector Vector Database**, **Docker Compose Support**.

Whenever we visit the Initializr, you'll click **GENERATE**, download the resulting `.zip` file, unzip it, and open the resulting project in your IDE, whatever IDE that is.

This project, and all the others in this book, will use the latest version of Spring Boot, which, at the time of writing, is 3.5.x, though preliminary tests with the 4.x line look fine, too.

Each of the things we selected as a dependency on the Spring Initializer will ultimately be a **Spring Boot Starter**. Starters do 95% of the heavy lifting in a given application. For example, you chose **Spring Web**; now, without doing anything more, your program will start with a full web framework *and* an embedded web server.

Unless otherwise noted, when I show you the source code, just paste the code after the very last `}` (curly bracket) in the code page that contains the `main` class (the one that contains the `public static void main` method).

In Spring, you can either annotate classes with *stereotype* annotations such as `@Component`, `@Service`, `@Controller`, or `@RestController` to tag them so that Spring discovers them and instantiates (while applying some sensible default conventions) a new instance, *or* you could provide a method annotated with `@Bean` that returns a constructed object, instantiating it whichever way is most desired.

Our map

We're going to look at how to pull together an interesting system using the wide and wonderful world of Spring. Let's take a look, in *Figure 1*, at the map before we begin our journey.

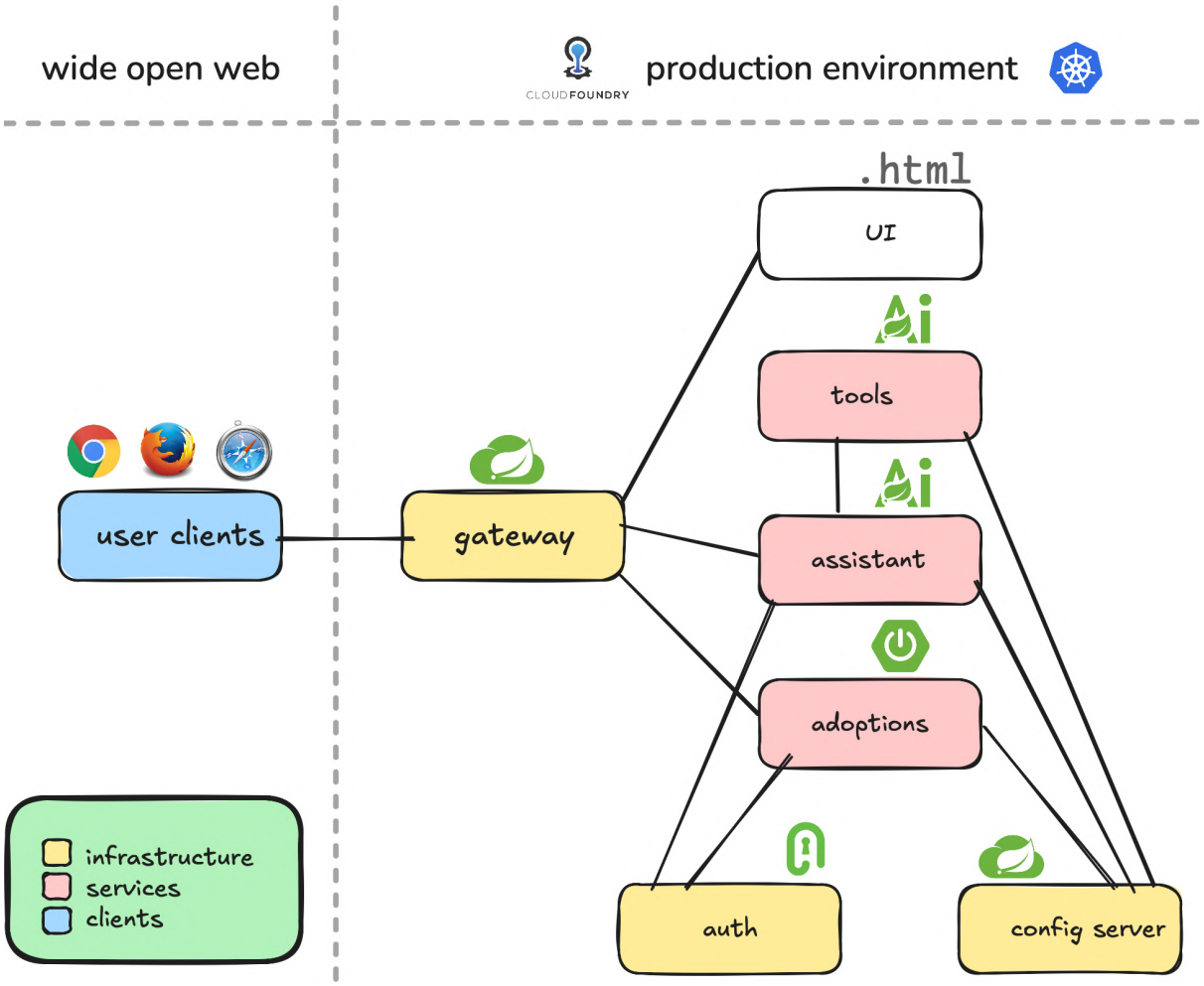

Figure 1: A map of what we're going to build

Let's expand on what each landmark in the map for our journey to production is:

- **Spring Cloud Config Server service** (`config` – shown as `config server` in the image): This trivial service will set up an HTTP API that we can use to then proxy requests being sent to a backing Git repository. Why would you do this? Well, all the other applications will have configuration values that they'd source from environment variables or their local `application.properties`, typically. We use the Spring Cloud Config Server to centralize all that configuration in one place and to store it in a version-controllable, centralized, auditable Git repository. We can also enforce consistent configuration across all the projects this way.
- **Spring Authorization Server service** (`auth`): We're going to look at OAuth, a protocol for centralizing and delegating authentication for any number of clients and services. We'll use the Spring Authorization Server to do the work of an OAuth authorization server. You could use something like Okta, Keycloak, Active Directory, and so on, as a stand-in. But why would you? This is *much* simpler, and for most organizations, it'll provide way more than enough bang for the buck!
- **Adoptions service** (`adoptions`): This will be the core API with our (contrived, trivial) "business logic." We'll investigate different technologies on top of HTTP: gRPC, GraphQL, Hypermedia, REST, and so on.
- **Gateway client** (`gateway`): Our `adoptions` service will need to be locked down and protected, so we will hide the service behind a Spring Cloud Gateway instance, which will proxy authenticated requests or initiate the "OAuth dance" to obtain a valid OAuth token.
- **Assistant** (`assistant`): Obviously, adopting a dog is important, but how do we find the dog in the first place? We'll need an AI-powered assistant to help people answer questions to find the dog of their dreams... or nightmares!
- **Tools** (`tools`): Our AI-powered assistant will need some tools that we'll expose using the MCP protocol.
- **UI** (`ui`): We'll need a user interface. This example is trivial, though this could be any static sort of UI artifact: React, Vue, Angular, Svelte, and so on. I don't want this example to be overly complicated, and we must be mindful of the resources of the internet and the planet as a whole, so we won't build out a full JavaScript application. There's just simply not enough space!

The data and its database

We're going to keep all the records related to dogs available for adoption in a PostgreSQL database, but we want support for the (optional) `vector` type later so that when we look at Spring AI, we have the ability to treat PostgreSQL as a vector store, which is a data structure that supports semantic similarity searches. This is why we chose **Pgvector Vector Database** on the Spring Initializr, but we have, in fact, a souped-up PostgreSQL database installation.

In the last section, when we incepted a new Spring Boot project on the Spring Initializr, we selected **Docker Compose Support**, which resulted in a `compose.yml` file being added to the root of the now unzipped project. Open up the `compose.yaml` file and make sure to open up the port being used for external connections. This way, you could use the `psql` CLI or whatever:


```
services:
  pgvector:
    # ... same as before
    ports:
      - '5432:5432'
    # ... same as before
```

Spring Boot will, if you ask it, automatically start up a working SQL database by running the `compose.yaml` file when you start the app. It can be made, if you ask it nicely, to execute `data.sql` and `schema.sql` when the application starts.

Let's ask it. Add the following to `application.properties` to specify our SQL database and to ask Spring Boot to execute our DDL `.sql` files.

```
spring.datasource.url=jdbc:postgresql://localhost/
mydatabase
spring.datasource.username=myuser
spring.datasource.password=secret
spring.sql.init.mode=always
```

It'll take a long time, as it is the first time since it has to pull a Docker image. You can disable this behavior if you have an existing SQL database to which you'd like to connect instead.

We're going to map some types to the SQL database and create a data access layer with the Spring Data JDBC ORM, like this:

For this to work, we'll need some valid SQL schema. Add the following two files under the `adoptions` module:

```
--src/main/resources/schema.sql
drop table if exists dog;
create table if not exists dog
(
    id          serial primary key,
    name        text not null,
    description text not null,
    owner       text null
);
```

Add some sample data to the `dog` table:

```
--src/main/resources/data.sql
insert into dog(id,name,description) values ( 3, 'Buddy',
  ' A Poodle known for being calm.');
insert into dog(id,name,description) values ( 63, 'Jasper',
  ' A grey Shih Tzu known for being protective.');
insert into dog(id,name,description) values ( 101, 'Nala',
  ' A spotted German Shepherd known for being loyal.');
insert into dog(id,name,description) values ( 61, 'Penny',
  ' A white Great Dane known for being protective.');
insert into dog(id,name,description) values ( 1, 'Bella',
  ' A golden Poodle known for being calm.');
insert into dog(id,name,description) values ( 91, 'Willow',
  ' A brindle Great Dane known for being calm.');
insert into dog(id,name,description) values ( 5, 'Daisy',
  ' A spotted Poodle known for being affectionate.');
```

```
insert into dog(id,name,description) values ( 95, 'Mia',
  ' A grey Great Dane known for being loyal.');
insert into dog(id,name,description) values ( 45, 'Prancer',
  ' A demonic, neurotic, man hating, animal hating, children
hating dog
    that looks like a gremlin.');
```

We've some business logic that we'll introduce once and reuse in subsequent examples:

```
@Service
@Transactional
class DogService {

    private final DogRepository dogRepository;

    private final DogFactsClient dogFactsClient;

    DogService(DogRepository dogRepository,
        DogFactsClient dogFactsClient) {
        this.dogRepository = dogRepository;
        this.dogFactsClient = dogFactsClient;
    }

    Dog byId(int id) {
       return dogRepository.findById(id).orElse(null);
    }

    void adopt(int dogId, String newOwner) {
        dogRepository.findById(dogId).ifPresent(dog -> {
            var updated = dogRepository.save(new Dog(dogId,
            dog.name(), newOwner, dog.description()));
            System.out.println("updated [" + updated + "]");
        });
    }

    Collection<Dog> all() {
        return dogRepository.findAll();
    }
}
```

This is our first business service. A business service is an accumulation of the code that corresponds to the business rules of our system. In this case, we've got methods to support coarse-grained behavior, such as finding a dog by its ID and adopting a dog. This brings us to a thought exercise you'll no doubt have encountered in philosophy in university: if you write business logic without providing a way for anyone to leverage it, did you *actually* write it? (the answer's *no*). Let's put our services on the web!

HTTP

Spring is a component model, but we don't really have time to get into all of it. What's important is that you understand you can create Java classes, annotate them, and perhaps specify some configuration in `application.properties`, and the framework will do most of the heavy lifting. The resulting boost in productivity is unparalleled.

Let's create a simple HTTP **controller** with Spring MVC:

```
@Controller
@ResponseBody
@RequestMapping("/http/dogs")
class MvcHttpController {

    private final DogService dogService;

    MvcHttpController(DogService dogService) {
        this.dogService = dogService;
    }

    @GetMapping
    Collection<Dog> all() {
        return dogService.all();
    }

    @PostMapping("/{dogId}/adoptions")
    void adopt(@PathVariable int dogId, @RequestParam
        String owner) {
        dogService.adopt(dogId, owner);
    }

}
```

You've got two endpoints: `/http/dogs`, and `/http/dogs/{dogId}/adoptions`. Easy! Try it out by pointing your browser to `/http/dogs` or submitting `POST` with `curl` or `http` to the `adoptions` endpoint.

REST with Spring HATEOAS

What we've built so far isn't *REST* as originally prescribed by Dr. Roy Fielding in his doctoral dissertation. Instead, it's just one level short of REST. If we're measuring by Dr. Leonard Richardson's *REST Maturity Model*, it lacks **hypermedia**. What's hypermedia? Just links. When the client makes a request, it receives the response along with contextual, navigable links appropriate for the response. The client can use the links to decide where to go next based on the state of the resource displayed to them. You see this all the time when your browser sends a `<link rel=".."..></link>` element down, telling the client where to find the styling for the document. Now, let's do that for the API. We'll use a project called **Spring HATEOAS**.

HATEOAS means **hypermedia as the engine of application state**.

We'll build a new Spring MVC controller, leveraging the new types brought in by Spring HATEOAS. First, let's look at the controller itself:

```
@RestController
@RequestMapping("/hateoas/dogs")
@EnableHypermediaSupport(type =
    EnableHypermediaSupport.HypermediaType.HAL_FORMS)
class HateoasMvcController {

    private final DogService dogService;

    private final DogModelAssembler dogAssembler;

    HateoasMvcController(DogService dogService,
        DogModelAssembler dogAssembler) {
        this.dogService = dogService;
        this.dogAssembler = dogAssembler;
    }

    @GetMapping
    CollectionModel<EntityModel<Dog>> all() {
        return dogAssembler
            .toCollectionModel(this.dogService.all());
    }
```

```
    @PostMapping("/{dogId}/adoptions")
    EntityModel<Dog> adopt(@PathVariable int dogId,
        @RequestParam String owner) {
        this.dogService.adopt(dogId, owner);
        var dog = this.dogService.byId(dogId);
        return dogAssembler.toModel(dog);
    }
}
```

The work of packaging up links and the payload is done by `ModelAssembler`, which we used a few times in the controller. Let's look at that implementation:

```
@Component
class DogModelAssembler implements
    RepresentationModelAssembler<Dog, EntityModel<Dog>> {

    @Override
    public EntityModel<Dog> toModel(Dog entity) {
        var controller = HateoasMvcController.class;
        var all = linkTo(controller).withRel("dogs");
        var adoptions = linkTo(methodOn(controller)
            .adopt(entity.id(), null)).withRel("adoptions");
        var selfRel = linkTo(controller,
        entity.id()).withSelfRel();
        return EntityModel.of(entity, selfRel, all,
            adoptions);
    }
}
```

Assuming you've added those classes, restart the application. Try it out by pointing your browser to `localhost:8080/hateoas/dogs` or submitting `POST` with `curl` or `http` to the `adoptions` endpoint.

GraphQL

REST works well, and it's a boon for the open web, but it's not necessarily the most useful (or efficient) protocol in which to build homogenous services. I think the honor of most useful goes to GraphQL.

GraphQL was developed at Facebook (now Meta) in 2012 and publicly released in 2015 with a specification, a reference implementation in JavaScript (the bad kind), and the GraphiQL in-browser console. It was born out of frustration with the limitations of REST APIs, especially in the context of mobile applications, where over-fetching and under-fetching data were common problems.

It requires some schema. Let's define some types mapping more or less to our `Dog` type, and a read operation (a *query*) and a write operation (a *mutation*):

```
--src/main/resources/graphql/schema.graphqls
type Dog {
    id :Int
    name: String
    owner: String
    description: String
```

```
    shelter: Shelter
}

type Shelter {
    city: String
}

type Query {
    dogs :[Dog]
}

type Mutation {
    adopt(id:Int, owner:String) : Boolean
}
```

We now need to map those methods to a controller:

```
@Controller
class GraphQlController {

    private final DogService dogService;

    GraphQlController(DogService dogService) {
        this.dogService = dogService;
    }

    @QueryMapping
    Collection<Dog> dogs() {
        return dogService.all();
    }

    @MutationMapping
    boolean adopt(@Argument int id, @Argument String owner) {
        try {
            dogService.adopt(id, owner);
            return true;
        }
        catch(IllegalArgumentException e) {
         //don't care..
        }
        return false;
    }
}
```

Set up the GraphQL `graphiql` console with some configuration in `application.properties`:

```
spring.graphql.graphiql.enabled=true
```

Restart the application. Try it out by pointing your browser to `localhost:8080/graphiql`, and there you can interact with the GraphQL service. Try this:

```
query {
  dogs { id, name }
}
```

We've only touched the surface here. What we've seen is little more than what we built with HTTP and REST. Where GraphQL really comes into its own is when you start defining **resolvers** (methods, in the Spring GraphQL world) to lazily and dynamically hydrate parts of the **graph** of objects required for a response. Imagine we wanted to return a `Shelter` object for each dog in the shelter, but we need to resolve it by calling another service. We could resolve it for each dog with an extra method added to the controller:

```
    // ...
    @SchemaMapping
    Shelter shelter(Dog dog) {
        return new Shelter(Math.random() > 0.5 ?
            "San Francisco" : "Paris");
    }
    // ...
```

In this example, we're just hardcoding some nonsense results, but we could do anything. We could delegate to another service and have it load up the results. The client need not know any of this! They just know they asked for records of the `Dog` type, and the `Dog` attribute has a `shelter` attribute, which is resolved by calling this method.

It does make you wonder, however, what happens if you wanted to load a *lot* of `shelter` references? This method will get called each time. Which is fine. Maybe. Except that you may end up with the dreaded *N+1* problem, where you have one load for all the `Dog` instances, and another call to load the `shelter` attribute for each of the dogs. GraphQL offers a convenient solution here, too, with the dataloader, and what Spring GraphQL supports as `@BatchMapping`:

```
@SchemaMapping
    Map<Dog,Shelter> shelter(List<Dog> dogs) {
        // ... call ONE service and get ALL the shelters
        // for ALL the dogs and return a map
    }
```

Again, the client doesn't change! And the client doesn't need to know anything. In our implementation, we progressively evolved the implementation in more and more efficient ways, all without ever breaking the schema contract with the client.

gRPC

If GraphQL is, in my humble opinion, the most useful way to build services, then gRPC is the most performant. Spring gRPC is a new project that integrates gRPC with the rest of the Spring ecosystem, including security, observability, and integration with Spring Boot's web stack. We added it with the Spring Initializr. We'll need to define some schema:

```
//src/main/proto/adoptions.proto
syntax = "proto3";

import "google/protobuf/empty.proto";

option java_package = "com.example.service.grpc";
option java_outer_classname = "AdoptionsProto" ;
option java_multiple_files = true;

message Dog {
  int32 id = 1;
  string name = 2;
  string owner = 3;
  string description = 4;
}

message DogsResponse {
  repeated Dog dogs = 1;
}

service Adoptions {
  rpc All (google.protobuf.Empty) returns (DogsResponse);
}
```

gRPC schema can feel like its own whole language because, wait for it, *it is*. As is GraphQL. So we won't get too much into this, but suffice it to say we're defining a type called `Dog`, a pluralized *(0..N)* container called `DogsResponse`, and then specifying a few options for how the gRPC `protoc` compiler is to transpile the schema into Java code.

Rebuild the project to trigger the transpilation that'll already have been set up for you when you created the project: `./mvnw -DskipTests package`. Make sure you add the generated code in the `target` folder to your project's source code roots, if necessary.

Now, you'll need to extend the generated superclass:

```
@Service
@Transactional
class DogsGrpcService extends
    com.example.service.grpc.AdoptionsGrpc.
        AdoptionsImplBase {

    private final DogService dogService;

    DogsGrpcService(DogService dogService) {
       this.dogService = dogService;
    }

    @Override
    public void all(Empty request,
        StreamObserver<DogsResponse>
        responseObserver) {
        var dogs = this.dogService//
            .all() //
            .stream() //
            .map(dog -> Dog//
                .newBuilder()//
                .setName(dog.name())//
                .setDescription(dog.description())//
                .setId(dog.id())//
                .build()//
            )//
            .toList();
        var dogsResponse =
            DogsResponse.newBuilder().addAllDogs(dogs).
                build();
        responseObserver.onNext(dogsResponse);
        responseObserver.onCompleted();
    }
}
```

What just happened? Who did this to me? My eyes! I know. Most of the code is just marshaling data from our data access layer to the generated types created by the transpiration process. You get used to it.

Anyway, restart the app and try it out:

```
grpcurl --plaintext localhost:8080 Adoptions.All
```

You should see all the `dog` entities printed to the console as JSON records.

Too easy!

Web clients

Getting online is easy with Spring. That's obvious by now. But half the fun of building on the web is reusing what's out there. They're called *web services* for a reason! If there wasn't a transitive web of services on which we could depend, and if we didn't think that was a good thing, we'd just call them silo services or something.

And Spring makes it trivial to *consume* network services, too. There's easy support for injecting gRPC stubs, for working with GraphQL, for example, but I want to focus on two "lower-level" clients: `RestClient` and declarative interface clients.

One builds upon the other, so let's take each in turn. Let's suppose we wanted to enrich our domain with knowledge of different dog breeds. And as it happens, there's a real-life web service for that called `dog.api`!

We're going to build two implementations of our client for `dog.api`. We only need one, but this exercise will let us explore a possibility – Spring's declarative interface clients – that you may not have known you had.

We'll define an interface and then provide two different objects that provide the capability described in the interface:

- A traditional implementation of the interface
- Using Spring's declarative interface clients

Let's look at the interface first:

```
interface DogFactsClient {

    BreedFacts getBreeds(int page);

    record BreedFacts(Collection<Breed> data) {
    }

    record Breed(String id, BreedAttributes attributes) {
    }

    record BreedAttributes(String name, String description,
        BreedLife life) {
    }

    record BreedLife(int min, int max) {
    }
}
```


There's only one method, but several Java records are used to map the structure of the returned JSON to our local Java domain.

RestClient

The first tranche of support for calling other HTTP services in the Spring ecosystem is the amazing `RestClient`. It's the foundational layer on which most everything else builds:

```
@Component
class RestClientDogFactsClient implements DogFactsClient {

    private final RestClient restClient;

    RestClientDogFactsClient(RestClient.Builder restClient) {
        this.restClient = restClient
                    .baseUrl("https://dogapi.dog/api/v1/")
                    .build();
    }

    @Override
    public BreedFacts getBreeds(int page) {
        return this.restClient//
            .get()//
            .uri("breeds")//
            .attribute("page[number]", page)//
            .retrieve()//
            .body(BreedFacts.class);
    }
}
```

Easy! In this case, with one fairly straightforward endpoint, this was perfectly reasonable. One imagines that with enough extra endpoints, however, the complexity would grow. Let's see whether we can use a little convention to simplify things.

Declarative interface-based clients

First, let's rework that interface. We don't need to change the shape of the interface itself, only to annotate the method:

```
    // ....
    @GetExchange("https://dogapi.dog/api/v2/breeds")
    BreedFacts getBreeds(@RequestParam(value =
        "page[number]") int page);
    // ....
```

For each new endpoint, we just need to annotate the appropriate methods. We'll have to define a factory (which we're calling `httpServiceProxyFactory`), but only once. And for each new interface we want to be turned into an actual client, we'd define a bean, as in the following for `dogFactsClient`:

```
@Configuration
class DogFactsClientConfiguration {

    @Bean
    HttpServiceProxyFactory httpServiceProxyFactory(
        RestClient.Builder builder) {
        return HttpServiceProxyFactory //
            .builder() //
            .exchangeAdapter(RestClientAdapter.
                 create(builder.build())) //
            .build();
    }

    @Bean
    DogFactsClient dogFactsClient(HttpServiceProxyFactory
        httpServiceProxyFactory) {
        return httpServiceProxyFactory.createClient
           (DogFactsClient.class);
    }
}
```

Amortized over several different endpoints, this is going to be *very* efficient and concise. But you do you! That's the point. You've got choices.

Security

Let's turn our eyes to a new service called `auth`, in which we'll set up a simple Spring Security application. We have two goals: to demonstrate how we might protect this solitary service and then to demonstrate how we might protect a system of services.

Return to the Spring Initializr and generate a new project with the following dependencies selected for `auth`: **Spring Web**, **OAuth 2 Authorization Server**, and **GraalVM Native Support**.

Download the resulting `.zip` file, unzip it, and then import it into your IDE. We'll need to change or define a few files:

```
#src/main/resources/application.properties
server.port=9090
```

Spring Security is all about authentication and authorization.

Let's lock down access to the following Spring MVC controller that's expecting `Principal` (an authenticated user):

```
@Controller
@ResponseBody
class HelloPrincipalController {

    @GetMapping("/")
    Map<String, String> hello(Principal principal) {
        return Map.of("message", principal.getName());
    }
}
```

We'll need some users. Add the following to the `AuthApplication` class. This will install an implementation of the `UserDetailsService` interface that keeps some users in memory. That's not ideal. Spring Security supports the **Java Database Connectivity** (**JDBC**, a standard for SQL database access)-based persistence as well as automatic password migration from one encoding to another, but we just don't have the space to get into that here.

Authentication answers the question of *who is making this request*:

```
    @Bean
    InMemoryUserDetailsManager
       userDetailsService(PasswordEncoder pe) {

        var rob = User.withUsername("rob")
            .roles("ADMIN", "USER").password(pe.
                  encode("pw")).build();
        var josh = User.withUsername("josh")
            .roles("USER").password( pe.encode("pw")).
                  build();

        return new InMemoryUserDetailsManager(Set.of(rob,
           josh));
    }
```

Authorization answers the question of *what permissions or rights this person has*. Configure Spring Security by adding this to the `main` class. We'll set up a few different means of authentication: form login, one-time tokens, and passkeys (or WebAuthN). Additionally, we'll define some authorization, requiring that all HTTP requests must be authenticated:

```
@Configuration
class AuthorizationServerConfiguration {

    @Bean
    SecurityFilterChain ourSecurityFilterChain(
    ConsoleMailOneTimeTokenGenerationSuccessHandler ott,
        HttpSecurity http) throws Exception {
        return http//
            .authorizeHttpRequests(ae ->
                ae.anyRequest().authenticated())//
            .formLogin(Customizer.withDefaults())//
            .oneTimeTokenLogin(o ->
                o.tokenGenerationSuccessHandler(ott))//
            .webAuthn(wa -> wa //
                .rpId("localhost") //
                .rpName("bootiful") //
                .allowedOrigins("http://localhost:9090",
                    "http://127.0.0.1:9090") //
            )
        .build();
    }
}
```

I'm sure you've seen form logins before, as in *Figure 2*. The user hits a page and gets a username and a password form. Authenticate, and they're done.

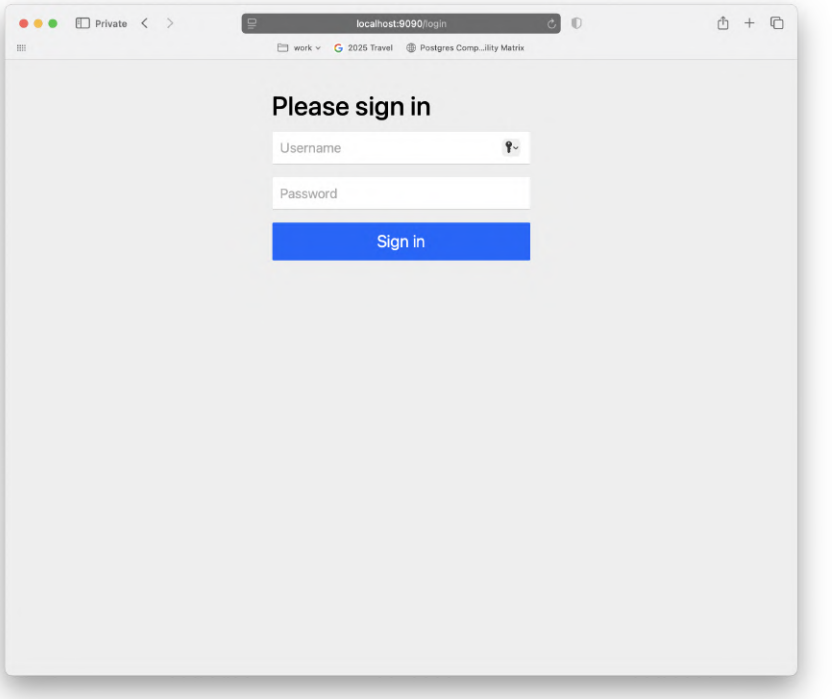

Figure 2: A simple form login

Toward a password-free future

The way we answer the question of authentication is by inspecting something that only the user could possibly have. For a long time, that *thing the user had* was a password. Passwords are awful. Don't take my word for it. Just ask the European Union and the United States government, both of which have several regulations on the books discouraging the use of passwords and recommending, at a minimum, a multi-factor security system.

The solution is simple: use passwords only when absolutely required. (But if you're going to use them, then at least make sure you're storing passwords encoded and supporting seamless password migration on authentication.)

Let's look at some ways to avoid passwords altogether.

One-time tokens

One-time tokens are sometimes called “magic links.” The user needs to only enter their username, and the system can send them an out-of-band password via some other medium that, by dint of possession, we assume means they are who they say they are. Mediums such as messaging apps or email.

That out-of-band communication is delegated to whatever implementation of `OneTimeTokenGenerationSuccessHandler` you provide. You could imagine using Twilio or Sendgrid, or something.

Let’s provide an implementation of `OneTimeTokenGenerationSuccessHandler` that simply prints the token to the console, which, for the purposes of our demo, is about as out-of-band as we’re going to get:

```
@Component
public class ConsoleMailOneTimeTokenGenerationSuccessHandler
    implements OneTimeTokenGenerationSuccessHandler {

    @Override
    public void handle(
        HttpServletRequest request,
        HttpServletResponse response,
        OneTimeToken oneTimeToken) throws IOException,
              ServletException {
            var msg = "go to http://localhost:9090/login/
                  ott?token=" + oneTimeToken.
                     getTokenValue() + " to login.";
            System.out.println(msg);
            response.setHeader(HttpHeaders.CONTENT_TYPE,
                MediaType.TEXT_PLAIN_VALUE);
            response.getWriter().write("you've got console
                     mail!");
    }
}
```

Restart the application and then bring up the login screen, and you'll see the new sign-in option, as shown in *Figure 3*.

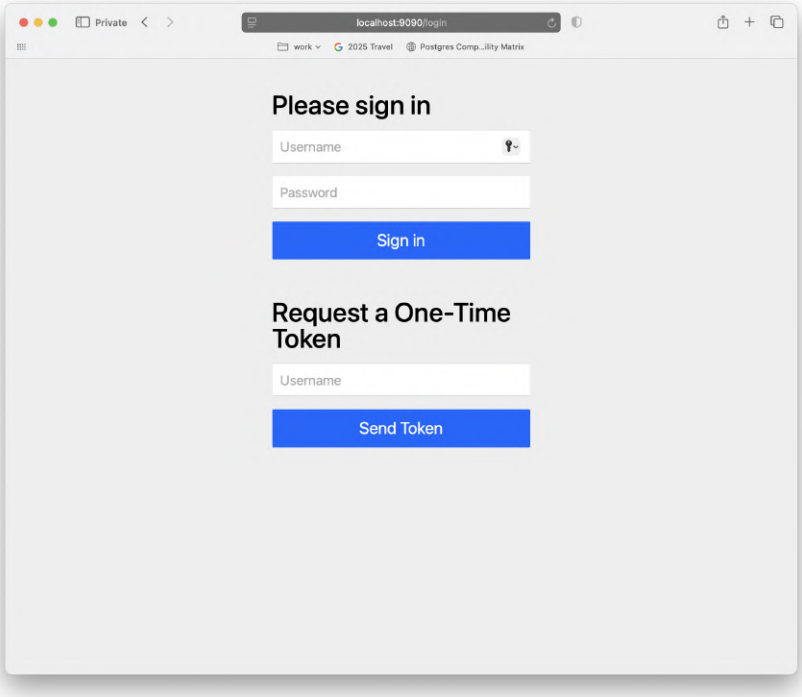

Figure 3: A new sign-in screen with one-time token support.

Passkeys

Passkeys is the brand name for a set of technologies and standards integrated across operating systems, browsers, mobile devices, and so on to allow the use of much less phishable factors (such as hardware keys, biometrics, etc.) to authenticate. It is the work product of the FIDO Alliance, whose membership (Apple, Google, Amazon, Netflix, various banks, etc.) is a who's who of people who have a literal vested interest in making commerce on the web as safe as possible. We set it up in our application in one or two lines of Java code.

First, you register a passkey, as shown in *Figure 4*:

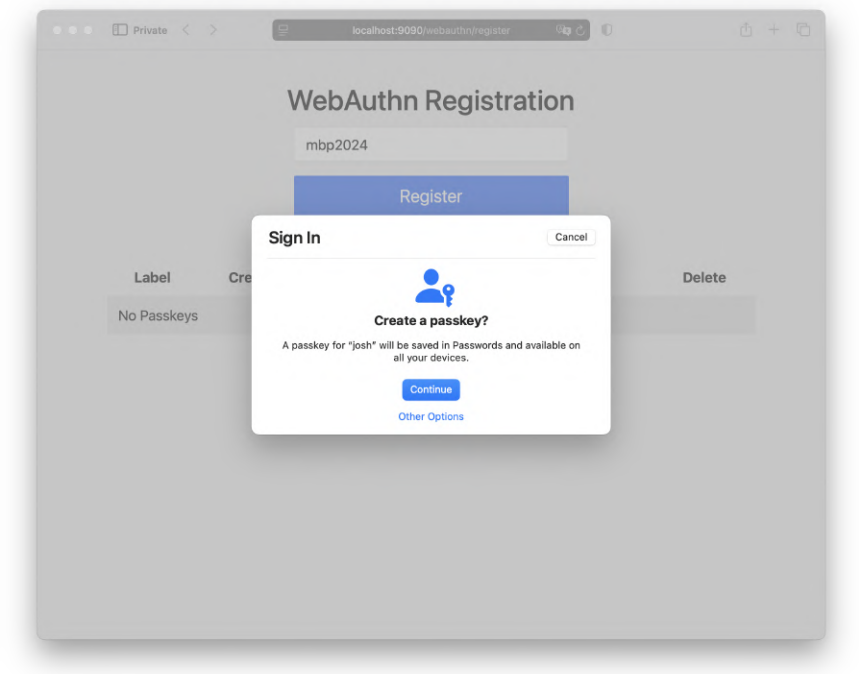

Figure 4: Registering a passkey

Then, you authenticate with that passkey, as shown in *Figure 5*:

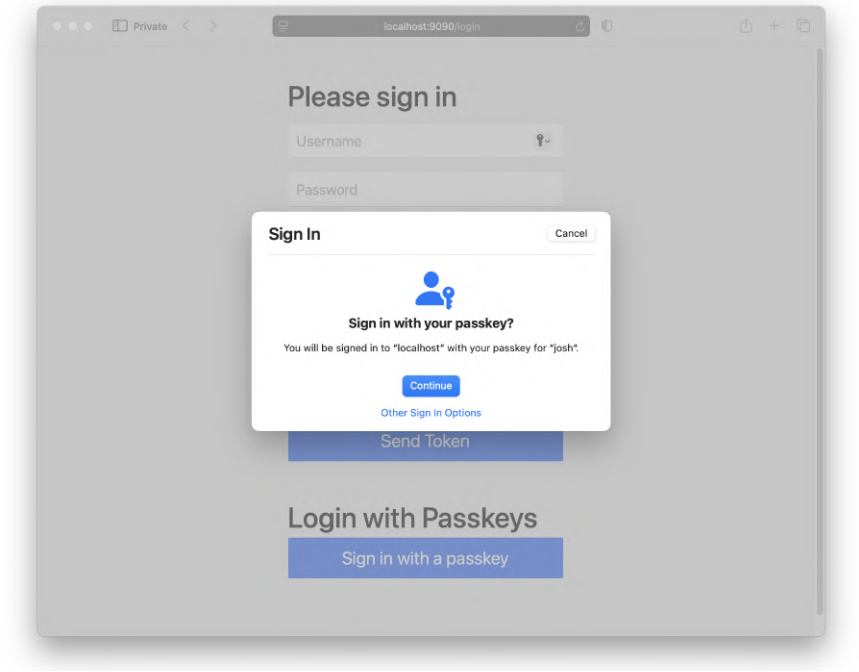

Figure 5: Authentication with the passkey

In these examples, I'm using macOS, which is tied to my iCloud identity. So I might use my finger reader on macOS, but then use FaceID on iOS. It's all tied to the same identity federated by iCloud. If I register a passkey with this application once, I can use it elsewhere. Similar convenience awaits those using Android or standalone password managers, too. Passkeys are a rare example of a technology that is both easier for the user *and* a better result. I love to see it.

Federated security

What about our `adoptions` module? How do we protect it? Easy! With OAuth! We'll need to do a few things to make this work:

- Transform our `auth` module into a standalone OAuth 2 authorization server with the Spring Authorization Server project
- Install a resource filter in the `adoptions` module
- Stand up a new module, an OAuth client, that will initiate the creation of an OAuth token in the event that it doesn't already exist

We've already got the Spring Authorization Server starter on the classpath for the `auth` module. Add this to the bottom of `HttpSecurityFilter`:

```
    // ...
        .with(authorizationServer(), as ->
            as.oidc(Customizer.withDefaults()))//
        .build();
    // ...
```

You can comment out `HelloPrincipalController` if you want, by the way.

An OAuth server will need to know what *clients* will interact with it, what kinds of permissions they'll want, to which URIs they'll expect to be redirected, and so on. You've probably seen this sort of thing on the web and the rut of **Sign In With Facebook**, **Sign In With Google**, and similar buttons. Every one of those pages is a client of Facebook, Google, and so on.

We can define them programmatically, source them from a SQL database, or, as we did earlier, define them in the source code, but this time, in `application.properties`:

```
spring.security.oauth2.authorizationserver.client.oidc-
    client.registration.client-id=spring
spring.security.oauth2.authorizationserver.client.oidc-
    client.registration.client-secret={noop}spring
spring.security.oauth2.authorizationserver.client.oidc-
    client.registration.client-authentication-
        methods[0]=client_secret_basic
spring.security.oauth2.authorizationserver.client.oidc-
    client.registration.authorization-grant-
types[0]=authorization_code
spring.security.oauth2.authorizationserver.client.oidc-
    client.registration.authorization-grant-
types[1]=refresh_token
spring.security.oauth2.authorizationserver.client.oidc-
    client.registration.redirect-
        uris[0]=http://127.0.0.1:8081/login/oauth2/code/
              spring
spring.security.oauth2.authorizationserver.client.oidc-
    client.registration.scopes[0]=openid
spring.security.oauth2.authorizationserver.client.oidc-
    client.registration.scopes[1]=profile
```

Restart the application. And that's that. We have a full-blown OAuth identity provider, such as Keycloak, Okta, Active Directory, and so on. Don't believe me? Hit `localhost:9090/.well-known/openid-configuration` and see for yourself.

Configuring the resource server

Return for a moment to the `adoptions` module, open up the `pom.xml` file, and add the following:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-
       server</artifactId>
</dependency>
```

This will install a filter around the `adoptions` service, rejecting all requests that don't have a valid and identifying OAuth access token. Who decides whether it's valid? The Spring Authorization Server, of course. Add this to `application.properties` for `adoptions`:

```
spring.security.oauth2.resourceserver.jwt.issuer-
uri=http://localhost:9090
```

This will tell the filter configured in the `adoptions` service to *ask* the newly minted Spring Authorization Server instance to either create a new token as required or validate a token given to it to confirm that it's authentic.

An OAuth client

So now, nobody without a valid token will ever get into that server. But where exactly *does* one get a valid token? Somebody's got to initiate the flow so that the client (the browser, app, whatever) ends up with a token that might be forwarded on to the backend `adoptions` module. That's the job of an OAuth client.

Hit the Spring Initializr (**gateway**: **Spring Web**, **OAuth 2 Client**, **GraalVM Native Support**, and **Gateway**).

Make sure it starts on a different port:

```
server.port=8081
```

We want to build an HTTP controller on port `8081` that makes a request to our backend `adoptions` service. While we're at it, let's introduce an alternative style of HTTP endpoint definition in Spring. Earlier, we looked at the *controller* style, but Spring also supports functional style registration, sort of like Express.js, Scalatra, Sinatra, Flask, and so on:

```
@Configuration
class ForwardingConfiguration {

    @Bean
    RouterFunction<ServerResponse> forwardingRoute(
        RestClient.Builder builder,
        OAuth2AuthorizedClientManager
          authorizedClientManager) {

        var requestInterceptor =
            new OAuth2ClientHttpRequestInterceptor(
                authorizedClientManager);
        var rc = builder.
           requestInterceptor(requestInterceptor).build();
        return route() //
            .GET("/", _ -> {
                var bodyType = new
                  ParameterizedTypeReference
                    <Collection<Map<String, Object>>>() {};
                var collectionOfDogMaps = rc //
                    .get() //
                    .uri("http://localhost:8080/http/dogs")//
                    .attributes(clientRegistrationId
```

```
                        ("spring"))//
                    .retrieve()//
                    .body(bodyType);
                System.out.println("got " +
                   collectionOfDogMaps);
                return ServerResponse.ok() //
                    .body(Objects.
                       requireNonNull(collectionOfDogMaps));
            }) //
            .build();
    }
}
```

Fundamentally, you register a bean of the `RouterFunction<ServerResponse>` type. For each request we get on port `8081`, a filter in the OAuth client code will either redirect the user to sign in with the Spring Authorization Server instance we set up on port `9090` or allow the request, which will then result in an authenticated call to the downstream `localhost:8080/http/dogs` endpoint.

The `/gateway-dogs` endpoint will remain completely inaccessible until the user has successfully authenticated. In a way, this code is acting like a proxy with a filter to ensure a valid access token.

If we're just going to forward and filter, then we might as well use a proper API gateway such as Spring Cloud Gateway, which very conveniently would be deployed for our application as a set of filters for the `RouterFunction<ServerResponse>` style of building HTTP endpoints. I'll leave that as an exercise for another day.

Let's wrap this up with some configuration that will tell the auto-configured OAuth client as to which OAuth client it is to behave. Recall that we set up the clients in the Spring Authorization Server. Now, we need to specify that we're acting as that client. Add this to `application.properties`:

```
# same as before ..
spring.security.oauth2.client.provider.spring
    .issuer-uri=http://localhost:9090
spring.security.oauth2.client.registration.spring.
provider=spring
spring.security.oauth2.client.registration.spring.client-
id=spring
spring.security.oauth2.client.registration.spring.client-
secret=spring
spring.security.oauth2.client.registration.spring
    .authorization-grant-type=authorization_code
```

```
spring.security.oauth2.client.registration.spring
    .client-authentication-method=client_secret_basic
spring.security.oauth2.client.registration.spring
    .redirect-uri={baseUrl}/login/oauth2/code/
       {registrationId}
spring.security.oauth2.client.registration.spring.
scope=openid
```

Try it out: hit `127.0.0.1:8081`. (Not `localhost`! We need two sets of cookies between the client and the Spring Authorization Server; if they're both `localhost`, then they'll clobber each other even if the port is different.) You should be redirected to the Spring Authorization Server, asked to log in, and then redirected *back* to the original application on port `8081`, which will, in turn, simply call the downstream service, passing along an access token. If you want to see that we've propagated the authentication of the request, you could change the implementation of the Spring MVC controller in our `adoptions` module to inject the current `Principal` and then print it out:

```
@Controller
@ResponseBody
@RequestMapping("/http/dogs")
class MvcHttpController {

    // same as before..

    @GetMapping
    Collection<Dog> all(Principal principal) {
        System.out.println("fetching all the dogs for "
            + principal.getName());
        return dogService.all();
    }

    // same as before..
}
```

So, this has worked, but our HTTP handler in the `gateway` module is just acting as a relay to a downstream service; this is precisely the kind of work that a proxy or a gateway could handle! Good point. Let's refactor the code to use Spring Cloud Gateway instead. And while we're at it, let's stand up a quick UI, too.

First, here's our *UI* code. It's a static HTML page that we're serving on port `8020`. It's a single, solitary page, but you could return any page, and any static resource: React, Vue, and so on.

Put this in a new directory called `ui` and name the file `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
       initial-scale=1.0">
    <title>Dog Data Table</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 20px;
            background-color: #f5f5f5;
        }

        .container {
            max-width: 800px;
            margin: 0 auto;
            background-color: white;
            border-radius: 8px;
            box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
            padding: 20px;
        }

        h1 {
            color: #333;
            text-align: center;
        }

        table {
            width: 100%;
            border-collapse: collapse;
            margin-top: 20px;
        }

        th, td {
            padding: 12px;
            text-align: left;
            border-bottom: 1px solid #ddd;
        }

        th {
            background-color: #4CAF50;
            color: white;
        }
```

```
        tr:hover {
            background-color: #f5f5f5;
        }

        .loading {
            text-align: center;
            margin: 20px 0;
            font-style: italic;
            color: #666;
        }

    </style>
</head>
<body>
<div class="container">
    <h1>Dog Registry</h1>
    <div id="content">
        <div class="loading">Loading dog data...</div>
    </div>
</div>

<script>
    window.addEventListener('load', async (e) => {
        const contentDiv = document.
           getElementById('content');
        const dogs = await (await window.fetch('/api/
           dogs')).json()
        contentDiv.innerHTML = '';
        const table = document.createElement('table');
        const thead = document.createElement('thead');
        const headerRow = document.createElement('tr');
        const headers = ['ID', 'Name', 'Owner',
          'Description'];
        headers.forEach(headerText => {
            const th = document.createElement('th');
            th.textContent = headerText;
            headerRow.appendChild(th);
        });
        thead.appendChild(headerRow);
        table.appendChild(thead);
        const tbody = document.createElement('tbody');
        dogs.forEach(dog => {
            const row = document.createElement('tr');
```

```
            // Create cells for each dog property
            const idCell = document.createElement('td');
            idCell.textContent = dog.id;
            row.appendChild(idCell);

            const nameCell = document.createElement('td');
            nameCell.textContent = dog.name;
            row.appendChild(nameCell);

            const ownerCell = document.createElement('td');
            ownerCell.textContent = dog.owner;
            row.appendChild(ownerCell);

            const descriptionCell = document.
                createElement('td');
            descriptionCell.textContent = dog.description;
            row.appendChild(descriptionCell);

            tbody.appendChild(row);
        });

        table.appendChild(tbody);

        contentDiv.appendChild(table);

    });
</script>
</body>
</html>
```

Now, run a little Python from the same directory as `index.html` to spin up an HTTP server to serve the contents of the HTML page so that we can proxy it:

```
python3 -m http.server 8020
```

Delete the forwarding `RouterFunction<ServerResponse>`. We're going to use the same functional style HTTP endpoint DSL, but we'll use Spring Cloud Gateway's intuitive extensions to it, instead, to cut short the work of proxying both the downstream API and the downstream UI code from the same host and port:

```
@Bean
    @Order(1)
    RouterFunction<ServerResponse> apiRoute() {
        return route("api")//
            .filter(tokenRelay()) //
            .before(stripPrefix())//
            .before(uri("http://localhost:8080/"))//
            .GET("/api/**", http()) //
            .POST("/api/**", http()) //
            .build();
    }

    @Bean
    @Order(2)
    RouterFunction<ServerResponse> staticRoute() {
        return route("cdn")//
            .filter(tokenRelay()) //
            .before(uri("http://localhost:8020/"))//
            .GET("/*", http()) //
            .build();
    }
```

We've defined *two* proxies, ordered relative to each other with the `@Order` annotation, so that Spring Cloud Gateway puts the greediest matcher (the one that matches all requests, `/*`) after the more specific ones.

We're using Spring Cloud Gateway's filters to specify downstream destinations and to say that we want to behave as a *token relay* – basically incepting the OAuth token if it's not already present in the request.

Try it out: if you go to `127.0.0.1:8081`, you should see an HTML table replete with the data it'll have loaded from our downstream resource server, where, again, you should see the authenticated `Principal`'s name printed out.

This approach is more secure – the client will never see the backend API or the HTML/JavaScript – and simpler, as there's no CORS to deal with, and no JWTs for the browser code to pass around. It's a win-win!

Building an assistant with Spring AI

At this point, we've done a good job of adopting the dog; we can adopt a dog in several different protocols with security aplenty. But how does one *discover* the dog of their dreams? You'd go to a shelter and interview the shelter about the prospective dog, wouldn't you? Let's build an assistant to help answer people's questions and shepherd people on their journey to dog adoption.

Hit the Spring Initializr and create a new module called `assistant` with **Spring Web**, **OpenAI**, **Model Context Protocol Client**, **Pgvector Vector Database**, **Spring Data JDBC**, and **GraalVM Native Support**.

Add the following values to `application.properties`:

```
#src/main/resources/application.properties.
server.port=8050
```

We're going to do something that you should never do, ever, not even when you are all by yourself at home and nobody's looking: we're going to copy and paste code. Namely, we're going to copy the `Dog` entity and `DogRepository` repository from the `adoptions` application. Paste them here:

```
interface DogRepository extends ListCrudRepository<Dog,
    Integer> {
    Dog findByName(String name);
}

record Dog(@Id int id, String name, String owner, String
    description) {
}
```

Likewise, we'll need to connect to our SQL database again, so add the following configuration to `application.properties`:

```
#src/main/resources/application.properties.
spring.datasource.url=jdbc:postgresql://localhost/mydatabase
spring.datasource.username=myuser
spring.datasource.password=secret

spring.ai.chat.memory.repository.jdbc.initialize-
    schema=always
spring.ai.vectorstore.pgvector.initialize-schema=true
```

These properties establish our connection to the SQL database, but they also help set up two things we need for two *advisors* that we'll explain momentarily. But first, let's look at the assistant itself:

```
@RestController
class DogAdoptionAssistant {

    private final ChatClient ai;

    DogAdoptionAssistant(
        QuestionAnswerAdvisor qa,
        DogRepository repository,
        PromptChatMemoryAdvisor memory,
        JdbcClient db,
        ChatClient.Builder ai,
        VectorStore vectorStore) {

        this.ensureVectorStoreDbInitialized(
           repository,db,vectorStore);

        var system = """
                You are an AI-powered assistant
                to help people adopt a dog from the
                adoption agency named Pooch Palace
                with locations in São Paulo, Tokyo,
                Singapore, Paris, New Delhi, Barcelona,
                San Francisco, and London.
                Information about the dogs available
                will be presented below. If there is no
                information, then return a polite
                response suggesting we
                don't have any dogs available.
                """;
        this.ai = ai
                .defaultAdvisors(memory,qa)
                .defaultSystem(system)
                .build();
    }

    @GetMapping("/{user}/assistant")
    String assist(@PathVariable String user,
                @RequestParam String question) {
        return this.ai
                .prompt(question)
                .advisors(a -> a.param(ChatMemory.
                   CONVERSATION_ID, user))
                .call()
                .content();
    }

    private void ensureVectorStoreDbInitialized(
```

```
            DogRepository repository,
            JdbcClient db,
            VectorStore vectorStore){
        var countOfRecordsInVectorStore = (int) db
                .sql("select count(*) from vector_store")
                .query(Integer.class)
                .single();

        if (countOfRecordsInVectorStore > 0)
            return;

        repository.findAll().forEach(dog -> {
            var content = "id: %s, name: %s, description: %s"
                    .formatted(dog.id(), dog.name(), dog.
                       description());
            var dogument = new Document(content);
            System.out.println("adding [" + content + "]");
            vectorStore.add(List.of(dogument));
        });
    }
}
```

This is a simple Spring MVC HTTP controller. Broadly, there are three acts: the constructor, the method that handles requests (`assist`), and a helper method (`ensureVectorStoreDbInitialized`) to make the setup a little clearer.

In the constructor, we build `ChatClient`, specifying some advisors that will be present on all requests and specifying a default *system prompt*. System prompts give our AI model a mission and parameters of what it's meant to be helping us do. In this case, we've told it to behave as though it's an assistant at a fictitious dog adoption agency with locations in multiple cities worldwide.

The Spring AI ChatClient

Most of the code revolves aroundSpring AI `ChatClient`, which is your one-stop shop for all interactions with`ChatModel`. Spring AI supports more than a dozen different `ChatClient` implementations, but in this case, we're using Spring AI's OpenAI support. You'll need to specify an API key in `application.properties` (`spring.ai.openai.api.key`) or, better, as an environment variable (`SPRING_AI_OPENAI_API_KEY`) so that Spring Boot will autoconfigure the `ChatModel` on which the `ChatClient` will depend. `ChatClient` instances are cheap; create as many as you need with as many configurations as you need.

Just like with `RestClient` and `JdbcClient`, the `ChatClient` provides a fluid DSL that should let you get a basically correct outcome with just autocompletion.

Advisors

The `assist` method takes an arbitrary question as a request parameter, sends it down to the `ChatClient` as the user prompt, and awaits its response. AI chat models have amnesia; they forget what's been said once they've said it, so you have to constantly remind them by sending a transcript of what's been said. This is a concern we address seamlessly by defining and injecting `PromptChatMemoryAdvisor` in the constructor. This, in turn, stores everything in JDBC.

We want our AI model to have access to the data in our SQL database to inform its responses. We don't need to send *all* of it, just that which might conceptually match. Remember, each interaction with the model has a cost, be it a dollars-and-cents cost or a complexity cost. To keep that to a minimum, we use the **retrieval-augmented generation** (**RAG**) pattern. When the program starts up, we read all the dogs from the SQL database and then write them out to our vector store implementation. Then, the second advisor, `QuestionAnswerAdvisor`, does the hard work of looking up the records from the vector store, finding only candidates that *might* plausibly be relevant to the request, and then stuffing the prompt with that data for final analysis by the AI model. Here are their definitions:

```
@Configuration
class AdvisorConfiguration {

    @Bean
    PromptChatMemoryAdvisor
    promptChatMemoryAdvisor(DataSource dataSource)
    {
        var jdbcChatMemoryRepository =
           JdbcChatMemoryRepository
            .builder()
            .dataSource(dataSource)
            .build();
        var messageWindowChatMemory =
           MessageWindowChatMemory
            .builder()
            .chatMemoryRepository(jdbcChatMemoryRepository)
            .build();
        return PromptChatMemoryAdvisor
            .builder(messageWindowChatMemory)
            .build();
    }
```

```
    @Bean
    QuestionAnswerAdvisor questionAnswerAdvisor(VectorStore
vectorStore) {
        return new QuestionAnswerAdvisor(vectorStore);
    }
}
```

Try it out! Launch the program and issue a request using, say, `http`:

```
http --form POST  :8050/jlong/assistant question=="Do you
have any neurotic dogs?"
```

If you call this endpoint, the AI should come back with a response indicating that it's found a dog named *Prancer* who matches our description. It's tough to show the result here because, since it's a conversational AI, the result could be ever so slightly different each time.

Tool calling

Nice! We've found the dog of our dreams, but how do we give our AI model a little executive agency? With tool calling, of course. Let's add a tool – a function that Spring AI can invoke at the behest of the AI model – to answer questions about when we might schedule an appointment to pick up or adopt a dog from a given location:

```
@Component
class DogAdoptionSchedulingService {

    @Tool(description = "schedule an appointment "+
         "to adopt a dog")
    String scheduleDogAdoptionAppointment(
            @ToolParam(description = "the id of the dog")
           int dogId,
            @ToolParam(description = "the name of the dog")
           String name
    ) throws Exception {
        System.out.println("scheduleDogAdoption"+
                "Appointment [dogId=" +
                dogId + ", name=" + name + "]");
        return  Instant.now().plus(3, ChronoUnit.DAYS).
            toString();
    }

}
```

The method itself will just return a default value of three days in the future. The important code is in the `@Tool` annotation, which describes the nature of this tool to the AI model. Configure it as a default tool on the `ChatClient` back in the constructor:

```
// same as before

DogAdoptionAssistant(DogAdoptionSchedulingService
    dogAdoptionSchedulingService,
    // ... same as before
) {
    // same as before
    ai
            // same as before
            .defaultTools(dogAdoptionSchedulingService)//
            .build();
    // same as before
}

// same as before
```

Restart the application. Ask the same question as before. Once it returns `Prancer`, then inquire about when you might be able to adopt him:

```
http --form POST  :8050/jlong/assistant question==
"Fantastic. When could I schedule an appointment to pick up
Prancer from the San Francisco location?"
```

The model should confirm there's availability and return a date three days in the future. Again, it's a conversational AI, so the response might vary from one run to the other.

Model Context Protocol

At this point, our AI model has access to both our data (thanks to our deployment of the RAG pattern through `QuestionAnswerAdvisor`) and our business logic, thanks to our deployment of tool calling. Remember, most organizations run on Java and Spring. Your business logic is most likely in Java and Spring, and it guards organizational data as part of its operations. It's *natural*, indeed *logical*, to want to simply hang some AI integration off the side of these code bases. You don't need Python to "do AI." Quite the contrary! 90% of the time, when people talk about "AI engineering," they're just talking about HTTP clients calling AI models' HTTP endpoints. It's integration code, and we in the Java community have long been the undisputed glue of the enterprise. This is our wheelhouse! We have a unique opportunity, my friends; let's pursue it.

That's not to say that there won't be other languages, runtimes, technologies, and tools that want to participate, however. And they should. That tool we built worked just fine, but it's hidden inside our Java code base. What if we could extract it and make it available to all AI-

powered systems? We will use **Model Context Protocol** (**MCP**), a new network protocol proposed by Anthropic in November 2024.

MCP is a specification for structured, interoperable communication between **machine learning** (**ML**) models, tools, and serving environments. It defines a standardized way to describe the context in which a model operates, such as its input/output schema, constraints, dependencies, and runtime expectations. By codifying this metadata, MCP enables model consumers and infrastructure (such as orchestrators, serving layers, or explainability tools) to programmatically understand how to invoke and reason about a model, reducing the reliance on brittle conventions or tribal knowledge.

MCP plays a crucial role in multimodel systems, especially where models are dynamically loaded, composed, or served across heterogeneous environments. It helps decouple model development from deployment and supports advanced use cases such as auditing, version tracking, A/B testing, and automated validation. By offering a common protocol for describing model behavior and requirements, MCP improves interoperability, governance, and maintainability across the ML lifecycle.

The protocol is another great example of winning with the web. Sort of. You see, MCP debuted in two flavors: STDIO-based and HTTP/SSE-based. STDIO assumes same-host connectivity; a *client* runs a *server* binary and communicates with it using standard in and standard out. Alternatively, a client can connect to a remote HTTP server using server-sent event streams. Spring AI supports building or consuming both varieties, but we're going to assume you're working with HTTP/SSE since that's more common and useful. MCP's simplicity is the killer feature. It's just HTTP. It's just the web. We're expanding the universe of possibilities for our AI models, and all we have to do is wield the web. It's hard to overstate how valuable this is. MCP is doing for AI models what language servers did for the commoditization of IDE plugins, except that was just for IDE plugins, and this is for *everything*. I've heard it described as the "AI USB-C moment."

Let's refactor our code base, extracting the tool into a separate module called `tools`, and we'll expose that capability to our assistant. Before we build the `tools` module, let's refactor `assistant`. We've already added the `McpClient` dependency to this code base, so there is no need to change dependencies. We won't, at the end of this process, have a local object but instead a client to talk to a remote resource, like this:

```
    @Bean
    McpSyncClient mcpSyncClient() {
        var mcp = McpClient
                .sync(HttpClientSseClientTransport
                .builder("http://localhost:8083").build())
```

```
                .build();
        mcp.initialize();
        return mcp;
    }
```

And then change the constructor like this:

```
    DogAdoptionAssistant(
            McpSyncClient mcpClient,
            // like before..
    ) {
        return builder//
                 // like before..
                .defaultToolCallbacks(
                    new SyncMcpToolCallbackProvider
                       (mcpClient))
                .build();
    }
```

Make sure to remove the reference to `DogAdoptionScheduler`.

Now, let's set up our `tools` service. Hit the Spring Initializr (**tools**: **Model Context Protocol Server**, **Spring Web**, and **GraalVM Native Support**).

Change the default port in `application.properties`:

```
server.port=8083
```

Now, cut and paste `DogAdoptionScheduler` from the `assistant` application to this code base. Then, define a single bean telling Spring AI's autoconfiguration what tools it should export as MCP services:

```
    @Bean
    MethodToolCallbackProvider
        methodToolCallbackProvider
            (DogAdoptionSchedulingService service) {
        return MethodToolCallbackProvider
                .builder()
                .toolObjects(service)
                .build();
    }
```

Start this application up and then restart the `assistant` application and retry the flow, inquiring about neurotic dogs and then asking for a date. You should see the dog printed in the console of the `tools` module!

And with that, you've now built an AI-ready application that uses MCP. MCP is a burgeoning new protocol, but it has given rise to a sort of Cambrian explosion of possibilities. There are *thousands* of different, useful MCP services out there! Every major platform (CloudFoundry, AWS, Google Cloud, Azure, GitHub, Microsoft 365, Salesforce, Zapier,

Heroku, etc.) supports working with their ecosystems in terms of MCP. This is the Star Trek voice-controlled computer we all dreamed of as kids. And it seriously changes the way we interact with systems. I'd go so far as to say that when you build your next product, you'd be well advised to consider the chatbox experience at the same time (or before!) you worry about Android, Apple Watch, Apple Vision Pro, and so on.

Once you've exported your MCP services, there are countless willing hosts that let you consume those services, including your IDE, Visual Studio Code, Cursor, Cline, Claude Code, and so on. They're all basically just chat boxes. Your users will be using one or more of those already. If not today, then tomorrow. Heck, even OpenAI, a competitor to Anthropic, has vowed to support it.

I mention all this because, somehow, it's *already* become a thing. In less than half a year, it went from protocol non grata to the torrential wave of possibility that we see today, propeled, I think, by the hype and hope of the WS-* SOA wave of the early 2000s – the idea that developers could become assemblers, just pulling down random APIs to build bigger and better. But, where that wave was repelled by a crush of technical issues and over-specification, MCP largely sidesteps them. Schema and WSDL? Just talk about the method in a description and hand it the method signature. The AI will sort it out. Orchestration? The AI will figure it out.

The only major gap in MCP at the moment is security, and that's being worked on. Soon, you'll be able to turn any MCP service into an OAuth resource server, just as we did earlier. Stay tuned!

Scalability

Throughout everything we've done, we've worked on the Servlet programming model, which assumes one thread per request. This isn't a problem until your requests start taking a long time and being greedy with the threads on which they're executing. One way to significantly alleviate the contention and dramatically improve your application's scalability is to use Java 21 and higher's **virtual threads**. This feature gives you the same scalability as you might expect from something like `async/await` in other languages, but without any of the endless, creeping verbosity of those other languages! For the most part, you can enable it in a Spring Boot application by adding the following configuration to your `src/main/resources/application.properties` file in any of the projects we've built thus far:


```
spring.threads.virtual.enabled=true
```

GraalVM

Every time we've initiated a new project on the Spring Initializr, we made sure to specify **GraalVM Native Support**. What is it? **GraalVM** is an OpenJDK fork from Oracle, adding new capabilities, including a *native image compiler*. As you know, Java already has a **just-in-time** (**JIT**) compiler that adaptively compiles frequently executed code paths into much faster native code, switching, in effect, that code from interpreted to machine code as the program is running. It's amazing, for sure... But it only kicks in after a good many runs; it may be that your hot-path code *never* gets translated into native code if you don't run it enough times. And all the while, you're not getting the benefits of that optimization. And that compilation isn't particularly RAM-friendly, after all. It *is* a full-blown compiler deployed alongside your production code in the same memory space.

Java is very dynamic. Every JVM program could dynamically load every class, so every class needs to be present. This means that your typical Java program has a lot of extra RAM in use that it doesn't actually need. And having access to all those extra types in the JVM is a security risk because if it were possible to trick some unsecured class into running arbitrary code, you'd end up with the Log4Shell issue that plagued the world a few years ago.

An **ahead-of-time** (**AOT**) compiler such as GraalVM fixes these issues:

- It dramatically reduces the surface area of your program
- It dramatically reduces its memory footprint and increases its startup performance
- It disallows ad hoc runtime reflection without a priori configuration

When you run the GraalVM native image compiler, you get a native binary that is operating system- and architecture-specific. It does *not* honor the old Java promise of write-once, run-anywhere. But.. who cares? I want this to run on a Linux machine in the cloud somewhere. So long as I compile it for that outcome, it'll be fine.

Your first steps with GraalVM

Let's suppose you had a class called `Main.java`:

```
public class Main {

public static  void main(String [] args) {
    System.out.println("Hello, GraalVM!");
    }
}
```

You would compile it in the usual way with `javac`:

```
javac Main.java
```

You could run this code on the **Java Runtime Environment** (**JRE**) like so:

```
java Main
```

Then, you could turn the resulting `.class` file into a native image, like so:

```
native-image Main
```

This might take a while, so take these few moments for yourself. Take a few good, deep breaths. Stretch your extremities. Try to have your eyes focus on something near and on something far for a few seconds each.

Once this finishes, you can run the newly compiled system- and architecture-specific native binary as you would any other native binary on your operating system. On macOS and Linux, you could run it on the shell: `./main`. On Windows, I suppose you'd most naturally double-click the resulting `.exe` file or invoke it from PowerShell.

This is the low-level approach, but we're working with Spring Boot and its prepackaged build files. And we did remember to select **GraalVM Native Support** back on the Spring Initializr, so...

Your first steps with GraalVM and Spring Boot

Let's compile our Spring Boot applications into a GraalVM native image. From the root (where the `pom.xml` file lives) of one of the projects we've created today, run the following:

```
./mvnw -DskipTests -Pnative native:compile
```

This should work fine out of the box most of the time. But not necessarily all the time. And for this, Spring has an AOT component model. That's right: Spring's lifecycle starts at compilation and continues until runtime. We don't have time to get into that here, but it's quite powerful.

If you've compiled a program, say, `tools`, then you can go to the `target` directory and find a binary named `tools` therein. Run it. It should spring to life faster than you realize what's even happened.

Building a Docker image

Naturally, you're going to want to get this thing to production and in production; be it CloudFoundry, Kubernetes, AWS, Google Cloud, or Azure, the currency of your application is a Docker image. Spring's build plugins for Apache Maven and Gradle both integrate Buildpacks, which, in turn, make it trivial to create a Docker image out of your program.

If you just want a Java program inside a Docker image, then do the following:

```
./mvnw -DskipTests spring-boot:build-image
```

If you just want a GraalVM native image inside a Docker image, then do the following:

```
./mvnw -DskipTests spring-boot:build-image
```

Either way, the plugins will run and dump out the name of the container it's just created for you. You can use `docker tag` and `docker push` to tag and push those images to the container registry of your choice.

Business use cases

I'm going to set up a duality here, and it's an oversimplification, I grant you, but it's helpful to frame my thinking. People use Spring because it's a framework for building frameworks, and they use it because it's a framework for building applications. I suspect latter is the vast majority of people who use it. But that's not to say there aren't some in the former.

Indeed, all the various Spring projects themselves in turn extend the Spring Framework and add even more value to the core Spring value proposition.

There are many reasons to use Spring and Spring Boot. Obviously, it offers an endless list of features supporting all manner of use cases on the server side. The list goes down to the floor and out the door! It is the richest integrated framework for building backend server-side applications, with features supporting all manner of use cases. Let's look at some of them:

- **Web applications babysitting databases**: Spring offers a comprehensive web stack that enables you to build browser-based applications with ease. It supports REST services, WebSockets, and more. It also provides an extensive set of connectors for virtually any datastore you can imagine working with.
- **Artificial Intelligence (AI)**: Do I really need to spell out AI? Lol. It's 2025. Of course not. We can take for granted in 2025 that folks know what AI is. You know what I can't take for granted? That folks know that Java is actually a fantastic choice for AI engineering. You see, when most folks talk about AI engineering, they're talking about getting data to and from a REST endpoint. There's nothing endemic to Python or that community. Java offers an excellent set of frameworks and tools, starting with Spring AI and agentic frameworks such as Embabel, which, of course, builds upon Spring AI.
- **Data integration**: Data is the lifeblood of a modern organization, and Spring offers exceptional capabilities with its projects—Spring Batch, Spring Data, Spring Integration, Spring Cloud Stream—to find, access, manipulate, and conduct that data.
- **Microservices and cloud native systems**: Spring offers integrations with all the hyperscaler clouds, most of which are maintained by the hyperscaler in question. It provides infrastructure to meet the needs of standard distributed systems, including centralized configuration, service registration, and discovery. It also includes support for patterns and primitives for working with distributed

systems—things such as circuit breakers, retries, stream processing, and so on.

- **Modular microservices**: Want to build a well-designed, well-scaled modular monolithic application? Spring Modulith can help here.
- **Security**: No app is production-worthy until it's secure. Spring Security has some of the best security integrations and features in one cohesive family of technologies, called **Spring Security**.
- **Excellent framework for building frameworks**: The people who build applications, by far, are the largest group. However, there are a good many who use Spring because it's a very flexible component model, and that component model makes it trivial for organizations to encode the domain of their business into the code itself. Remember, a framework is "open for extension, but closed for modification." In this case, Spring is actually open source, so you can modify the code if you want. But the point is that you don't have to. There are extension hooks aplenty that make it trivial for you to interpose yourself wherever you need to in the Spring lifecycle. Let's look briefly at some of the extension planes that Spring users might integrate with in an effort to build tooling and abstractions that right-size for their organizations.
- **Autoconfiguration and Spring Boot starters**: The key to the concept of Spring Boot is the idea of autoconfiguration. Its configuration lives in a library (a Spring Boot starter) and gets evaluated when the application starts up, usually just by being on the classpath. This means that you can layer new capabilities into a Spring Boot application simply by adding dependencies to the classpath. A lot of the time, the code that's run at startup is only intended to run if some conditions are proper, so Spring Boot provides a whole subsystem for evaluating those conditions and using those conditions to gate new features from being added to the system. One condition might be the presence or non-presence of a given class in a third-party library. A starter is meant to bring in both the required libraries to support a given feature and the autoconfiguration to react to the presence of those libraries.
- **Spring Initializr**: Spring Initializr—or `start.spring.io`—is a place to start new Spring Boot projects. You get there, specify the parameters of your new Spring Boot project's build, and you're off to the races! It'll give you a new project with a working Maven or Gradle build, plus the required Spring Boot starters. If you're a large enough organization, it's entirely conceivable that you have built up frameworks on top of Spring Boot, published them to your company's internal (or indeed Maven Central, externally) artifact repository, and now want to socialize those dependencies so that

developers can build them into their applications internally. This might be an optional or required step for applications to progress to production. Well, good news! Spring Initializr is actually just some autoconfiguration and some starters, and they're open source. You could build your own Spring Initializr. Fork our Initializr project, change it to include and account for your particular dependencies and artifact repositories, and required defaults, and off you go! Another thing you might do is remove artifacts that are supported on the canonical `start.spring.io` that make no sense in your organization. You may not need Azure if your shop is using Google Cloud, or vice versa.

These create the means of action, distribution, and activation. But what can a starter do, exactly? Anything you want! The idea is that they usually install objects in your application that do things you want it to do. Things that you can then inject or things that, once wired together in the right way, result in something interesting, such as a web server or a Kafka listener being set up.

These starters could contribute beans that implement several key extension plane interfaces in the Spring ecosystem. This is where the real power in Spring is. It's a framework, and so you can *see* all the objects in your application. There are natural extension planes by which to augment Spring's and your application's capabilities. Let's look at some of them.

- **BeanPostProcessor**: Implementations of this interface are called back during the initialization phase and given a chance to change any other object before it's made available for injection.
- **Service loaders**: `spring.factories` is a classpath-wide registry (in `META-INF/spring.factories`) that Spring uses to discover and instantiate framework extension points (e.g., Boot auto-config, `ApplicationListeners`, `EnvironmentPostProcessors`). Each JAR can contribute entries; Spring merges them at runtime. There are tons of well-known keys for which you might contribute implementations. Some common ways include `EnvironmentPostProcessor`, `FailureAnalyzer`, `ApplicationListener`, and others.
- **EnvironmentPostProcessor**: `EnvironmentPostProcessor` makes it easy to let Spring applications *see* keys and values from configuration sources, such as JNDI, LDAP, property files, HashiCorp Vault, Spring Cloud Config Server, YAML files, and so on. Your organization might have a property source that Spring doesn't know how to talk to, but it's trivial to add support for it by implementing this interface.

- **Spring Boot Actuator**: Observability is a massive part of the reason people use Spring Boot: it's production-ready out of the box. It includes a set of managed endpoints that surface information about the application, such as health indicators, metrics, and software bills of materials. All of this observability is extensible. Want your cloud platform, such as Cloud Foundry or Kubernetes, to flag the service as failed when a particular subsystem of your organization goes down? Contribute a `HealthIndicator`. Easy!
- **A very flexible component model**: Even the Spring component model itself is designed to be flexible. It's very natural to use Spring support for meta-programming to create new annotations that have the behavior of a Spring application, but have stereotypical (in the UML sense) names that better identify the role of a component in an application. You can even have the framework automatically map attributes specified on these annotations and map them to the meta annotation. Suppose your organization wants to introduce a new kind of component to support the model-view-presenter pattern. Let's call this `@MvpController`. You could create such an annotation and then, on the body of the annotation, add Spring's `@Component` annotation. Now you don't need to specify `@Component` whenever you specify `@MvpController`. Neat, eh?
- **POJOs – Plain Old Java Objects**: One thing you'll notice is that the dependency injection container is meant to be as unobtrusive as possible. That's the whole point. Annotations don't change the type signature of your code. Unless you are specifically interfacing with Spring infrastructure, you won't have to implement an interface. We want your code to remain as you intended it, with as minimal an imposition from Spring itself.

  Spring bends, but doesn't break. We've looked at a *smörgåsbord* of amazing features in the Spring ecosystem, but the important takeaway here is that Spring is extensible. It bends, happily and readily, to any workflow.

Let's look at some examples!

Amazon's gambit—scale or stall

Most organizations would be well advised to start with a *modular* (that is, designed from the jump for decomposition) monolithic application and then, if and only if it becomes too unwieldy, spin off into separate services.

Modularity is a technical term, but it's designed to serve a social and political end. What we want are small, self-contained, modularized components in a code base on which small, self-contained, modularized teams can work. The fewer interdependencies between modules, the better. After all, these dependencies represent de facto contracts, and the more of those in a code base, the more work it is to effect changes without careful coordination.

Some people choose microservices—small, self-contained, network services—over modular code, but for largely the same reasons.

Knowing when and why to factor out a code base into smaller components, modules, or services is a very important lesson, and one that Amazon had to learn to survive.

In the early 2000s, Amazon had a big problem. They were too successful! They were branching into new domains with new diversified business interests—e-commerce, logistics, payments, recommendations, and so on, and their monolithic application was becoming a bottleneck.

They had one monolithic code base, and this meant that any changes to the code base required coordination across hundreds of engineers, slowing delivery. The monolith was also a single point of failure; if anything went wrong anywhere, it meant the whole system was down, and with it, perhaps, the entire company. Of course, one of the biggest problems with a monolith is that it constrains technology choices. Scaling a monolithic application is difficult, too, because you have to scale the whole application even if only one part, one use case, of it is failing to keep pace. Amazon had thousands of developers worldwide, and scaling that on one code base would have meant constant issues around code ownership, deployment, tests, and so on.

Amazon moved to the microservices architecture because it allowed small, independent teams ("two-pizza teams") to own a service end to end, from development to deployment. This aligned with Jeff Bezos' original philosophy: decentralized, autonomous teams could move faster. And naturally, as each service was a standalone, self-contained thing, teams were free to pick the right technologies for the job. If one service failed, then only part of the overall system would be inaccessible. Microservices were a great way forward, but they represented the refinement of decades of ideation around scaling organizations.

Conway's law, named after computer scientist Melvin Conway, says that "organizations which design systems (in the broad sense used here) are constrained to produce designs which are copies of the communication structures of these organizations." Another way I've heard it said is, if you have two teams building a compiler, you'll end up with a two-pass compiler.

Put another way, if there's dysfunction in your organization, it will be reflected in the software that this organization builds. And if you want to fix that dysfunction, then you need to fix the organization.

Microservices are a nice way to do this. Divide larger code bases into smaller self-contained services, and then build teams around those smaller code bases. The communication for everybody working on that one service flows freely. And, even better, there's a sort of encapsulation that happens. It's impossible to inadvertently depend on a type if there's a network boundary in the way, after all!

This section doesn't speak, specifically, to whether Amazon uses Spring Boot or not, though it certainly seems to be a common requirement for a lot of job postings of theirs. I hope, instead, to remind you of what a holistic look at an organization can do to drive change and scale. These winds of change are often why people start embracing patterns and practices such as cloud native computing and microservices. While there are many benefits to that approach, there is also a lot of complexity that falls out of it. Organizations need a framework that allows them to automate away and factor out as much complexity as possible, and for many, that framework is Spring Boot.

Let's look at a few examples.

Netflix—scalable teams, scalable systems

Netflix is a subscription-based, on-demand streaming service that enables members to watch TV shows, films, anime, documentaries, and even games across internet-connected devices such as TVs, video game consoles, tablets, and phones. You can even run it in your browser! You know... if nothing else worked. Look! I don't have to tell you what Netflix is. As a consumer-facing company, they're one of the darlings of the first quarter of the century. Their brand is so iconic that it's literally part of the popular vernacular in specific contexts.

Netflix is well known worldwide for its products and productions. But did you know it's also well known in the technology space? In 2010, as the world was grappling with the questions and novelty of the cloud, Netflix was experiencing its own renaissance. It realized that its architecture, like Amazon's, had grown too large. So, forearmed with the lessons of

Amazon and the wisdom of its leaders, including veteran Adrian Cockroft, it started forging its way forward. It was rebased on top of AWS, and began grappling with what it meant to build systems and services for organizational and, importantly, runtime scale.

Needless to say, it has quite a bit of scale!

As of August 2025, Netflix had approximately 301.6 million global subscribers, solidifying its position at the top of the streaming industry. In the second quarter of 2025, the company generated $11.08 billion in revenue, marking a robust 16% year-over-year growth, attributed to strategies such as ad-supported tiers, password-sharing controls, and live content offerings. Additionally, 2024 saw Netflix's highest-ever quarterly subscriber gain, adding 19 million new subscribers in Q4, boosted in part by viral hits such as Squid Game and its expansion into live programming, including sports and events.

Netflix has a strong philosophy of individual responsibility. You don't have to use a particular technology, but you will be asked to pick up the phone if something goes down in the dead of night.

It's not like every team has to reinvent the wheel, though. There are some conventions and infrastructure.

Services would need to find and connect to other services, even if application instances in the cloud came and went or DNS was unreliable. So, it built a service registry called Eureka. This service registry, like other popular tools such as Apache Zookeeper and, later, HashiCorp Consul, made it easy to register and discover other services with nearly no latency dynamically.

If the registry contains the listings of all the services in a system, then the thing that connects to the registry might have its own designs on which particular instance of a given service to route to. Netflix built a library called **Ribbon** to support client-side load balancing.

Netflix quickly realized that it'd need to centralize configuration values—keys and values for things such as port, database credentials, and so on, so it built a configuration tool called Archaius, which allowed applications to access static and dynamic configuration sources quickly and easily.

The network may be the computer, but the computer isn't always reliable. Things might take too long, or error out, or in some way impede progress in a system. Netflix built a library called Hystrix to wrap fragile code paths in *circuit breakers*.

Netflix embraced the GraphQL protocol, originally a project from Meta, as its communication technology. It even built up middleware called Netflix DGS. When the Spring team announced the Spring for GraphQL project, we were delighted to work with the Netflix DGS team so that, eventually, the DGS story would be a natural layer on *top* of Spring GraphQL.

Netflix built up a good many different backend services, and those services served all clients. But not all clients—TVs, phones, tablets, cars, and so on—are created equal. Some have nuances, security issues, and protocol requirements to bridge the frontends with the backends. These bridges were proxies, powered by Netflix's Zuul. (Get it? From Netflix? The gatekeeper?) Zuul is a scriptable, dynamic API gateway.

Netflix did the world a big favor by open sourcing so much of this infrastructure. A lot of this code was eventually the foundational layer for the work that would become Spring Cloud.

Spring Cloud has grown by leaps and bounds since then. It provides a standalone configuration server called the Spring Cloud Config Server, which is well recommended over the Archaius alternative, though integration with Archaius is still possible. Spring Cloud has a load balancer abstraction, for which Ribbon is but one implementation; a DiscoveryClient abstraction, for which Eureka is but one implementation; and an API gateway called Spring Cloud Gateway, which more than supersedes what Zuul offered.

As if inspired by the M.C. Escher drawing of a hand drawing itself, eventually Netflix rebased some of its work on top of Spring Cloud. Indeed, these days, most Netflix backend services are powered by Spring Boot- and Spring Cloud-powered.

- Some (perhaps speculative) information on Netflix subscribers: `https://www.demandsage.com/netflix-subscribers/`
- What is Netflix? `https://help.netflix.com/en/node/412`
- Netflix OSS and Spring Boot, Coming Full Circler: `https://netflixtechblog.com/netflix-oss-and-spring-boot-coming-full-circle-4855947713a0`
- Build GraphQL Services With Spring Boot Like Netflix (SpringOne 2024): `https://www.youtube.com/watch?v=DYZEkOsiPY0`

PayPal—when it absolutely, positively has to work

Paypal's a darling of the online commerce world and literally helped define the genre. It's also a large company for which, more than most, time literally is money. PayPal's architecture has grown significantly over the years. But by 2016, it was already talking about the challenges it was facing and trying to confront. PayPal needed to move away from a monolithic application structure to a more modern, flexible microservice architecture. This transition aimed to enhance scalability, reliability, and development speed. The goal was to make it easier for development teams to work on different parts of the application independently. PayPal chose Spring Boot as the foundational technology for its new architecture. Spring Boot simplifies the creation of production-ready, standalone applications with minimal configuration, allowing them to iterate rapidly.

As with many large organizations, it ended up building its own framework on top of Spring Boot called Raptor (`https://www.youtube.com/watch?v=khzC-VwpFVM`). (Can we agree that Raptor has to be the coolest-sounding name ever?) Raptor provided many out-of-the-box features for its particular use cases. It's an internal Java framework. It includes support for various applications and services, such as the following:

- RESTful services for APIs
- Batch applications for handling large, asynchronous tasks
- Message daemon applications for processing messages

Raptor builds upon Spring Boot to pull in support for all the various useful libraries, such as Spring Batch and Spring Integration. It also built its own custom Spring Boot starters to provide easier integrations for libraries that Spring Boot doesn't support out of the box.

PayPal also developed its opinionated path to production with a custom CI pipeline. This allowed it to iterate more quickly. As part of this, it came to really appreciate Spring Boot's self-contained `.jars` files; this made deployment a lot easier.

By 2019, its scale was growing even further, and it wanted to handle what scale it did have with even more ease, so it moved to Spring's reactive support, underpinned by the fabulous Reactor library. Reactive programming is a style of programming designed to free up clock cycles on the CPU. It inverts the usual control flow of programming; instead of "pulling" data from various sources and waiting until the source has been drained, reactive programs have the data "pushed" to them if and

when time permits. This means that no time is spent waiting for data that hasn't materialized yet, and thus fewer CPUs are blocked by waiting threads, allowing access to the CPU by other components in the system.

PayPal thrives today as one of the largest payment processors in the world.

Intuit—scaling money management for the masses

Intuit is best known as a world-class financial software company, serving tens of millions of people with products such as TurboTax, QuickBooks, Mint, Credit Karma, and Mailchimp. At its scale, the stakes are high: reliability, security, and developer productivity are not just nice-to-haves—they're essential to maintaining customer trust and delivering value quickly.

On a recent episode of A Bootiful Podcast, Katie Levy from Intuit described what it's like to drive the adoption of modern technologies such as Spring Boot, Kotlin, and Android inside an environment with plenty of legacy systems. Her career journey—beginning in Android development on TurboTax and TaxCaster apps—gave her an appreciation of how mobile requirements ripple back into platform and backend choices.

For Intuit, Spring Boot powers much of its backend services. It enables it to build cloud-native, microservice-based systems that can scale to support tax season surges, all while handling sensitive financial data securely. Spring Boot also helps Intuit reduce boilerplate and improve maintainability—critical when many different teams need to collaborate on a massive code base.

It has even developed internal libraries such as RWebPulse to streamline reactive HTTP client integrations, making Spring Boot services more robust with built-in exception handling, retries, and runtime configurability.

Katie highlighted Kotlin's growing role at Intuit—first on Android, but increasingly on the server side. Kotlin brings safety features such as nullability checks, concise syntax, and reduced boilerplate compared to Java. Just as importantly, it plays nicely with existing Java code, allowing gradual adoption.

Kotlin also makes **functional programming** (**FP**) more accessible. Katie herself is passionate about FP, though she noted that introducing FP concepts to teams steeped in object-oriented Java can be challenging. The key is incremental adoption: weaving in functional patterns where they make sense, while showing teams the practical benefits.

Technology adoption at a company the size of Intuit isn't just about tools—it's about people. Katie described how success depends on building internal learning communities, sharing examples, and creating visibility across teams. It requires empathy: addressing not only the features of a new language or framework, but also the concerns people have—migration costs, confidence in maintaining existing systems, and the risks of change.

This cultural layer is what allows Intuit to keep evolving. By blending world-class products with a thoughtful engineering culture, they're able to adopt new technologies in ways that improve long-term maintainability, correctness, and developer velocity—all while serving customers who depend on them during life's most financially stressful moments.

Intuit's story is one of incremental, thoughtful modernization. With Spring Boot at the core, Kotlin increasingly in play, and a culture that balances technical innovation with empathy, they're shaping a future where reliability and developer happiness go hand in hand.

- https://github.com/intuit/RWebPulse
- https://spring.io/blog/2020/07/17/a-bootiful-podcast-intuit-s-katie-levy-on-spring-boot-kotlin-android-and-more
- https://www.britannica.com/money/Uber
- https://www.codingshuttle.com/blogs/why-do-the-big-tech-companies-choose-spring-boot-to-build-their-microservice-application/
- https://bytebytego.com/guides/uber-tech-stack

Alibaba—Spring at China scale

Founded in 1999, the Alibaba Group has grown into a vast multinational conglomerate that powers a comprehensive digital ecosystem, encompassing its core e-commerce platforms, such as Taobao and T-mall, as well as its robust cloud computing, logistics, and fintech operations. As a company that recently reported quarterly revenue of over $38 billion and is investing more than $50 billion in its cloud and AI infrastructure, Alibaba operates at a scale that demands a highly efficient and scalable technology stack. To meet this challenge, Alibaba has adopted and heavily invested in Spring Boot, building a substantial portion of its microservices on the framework.

When I talk about Alibaba here, broadly, I'm referring to a web of companies and properties such as TaoBao.com, Alibaba.com, AlibabaCloud.com, Ant Financial, Alipay, and others that are loosely connected and share some foundational technology stack choices.

Alibaba's strategic use of Spring Boot is exemplified by its Spring Cloud Alibaba ecosystem. This initiative provides a seamless way for Spring-based applications to integrate with Alibaba's middleware and cloud services. Through official "starters" and auto-configuration, developers can use a familiar Spring paradigm to leverage Alibaba's proprietary technologies for critical microservices functions. These include using Nacos for dynamic service discovery and centralized configuration, Sentinel for traffic shaping and circuit breaking, and RocketMQ for event-driven messaging.

The company further supports this integration with its **Enterprise Distributed Application Service** (**EDAS**), a platform that provides full stack management and monitoring for Spring Boot applications deployed on Alibaba Cloud. The commitment to a Spring-centric ecosystem is a key business decision, ensuring that developers can rapidly build and deploy production-grade microservices that feel "native" to Alibaba's platform. This strategic alignment extends to new areas, such as Spring AI Alibaba, an effort to provide developers with the tools to build large language model applications on the Alibaba platform, demonstrating Spring's role as a foundational technology for Alibaba's future-facing initiatives in artificial intelligence and beyond.

I first learned about Alibaba in 2011. It'd just gone through its annual shopping festival 11/11 (or "Singles' Day"). You can think of it sort of like the "Cyber Monday" or "black Friday" shopping holidays here in the West. It's a time when e-tailers flood the market with promotions and discounts. In China, Alibaba was the first and biggest to lead the charge for this festival. And lead it did. In 2012, on its November 11 event, it sold more than $3 *billion* in products in a single 24-hour period and handled 147 *million* user visits during that same time. That's an *incredible* scale. For reference, in the West, black Friday sales that year for *all* the major online e-tailers on Black Friday were also around $1 billion USD (`https://www.searchenginewatch.com/2012/11/26/black-friday-e-commerce-sales-set-1-billion-record-2012-holiday-online-sales-strong/), and for Cyber Monday (again, for all e-tailers`), at around $1.2 billion (`https://thenextweb.com/news/comscore-black-friday-online-sales-hit-1-2-billion-with-amazon-the-top-retailer`).

I just knew I needed to learn more, so I jumped on a plane and sat down with some of the nice engineers at Alibaba. Even back then, years before we announced Spring Boot, they'd developed remarkable frameworks to handle building systems, services, and remote procedure calls. One framework was called SoFA, and another was called Dubbo. Dubbo has grown so popular that it's now an Apache project! I documented this, and more, in this blog from early 2013.

Obviously, Spring Boot arrived, along with starters for various bits of infrastructure, both internal and external. Dubbo got Spring Boot starters, too. It even led the charge in developing a Spring Boot and Spring Cloud-style set of integrations for its cloud platform, called Aliyun (or *Alibaba Cloud*). Look at its open source projects and you'll see Spring Cloud Alibaba, a set of integrations for Aliyun or locally-hosted infrastructure, a hyperscaler like Amazon Web Services or Google Cloud that is very popular in China. Aliyun, in turn, can host and scale up a lot of the typical infrastructure with which you're no doubt familiar – popular message queues and databases and object storage, for example – *and* a fair bit of infrastructure with which you might be less familiar. Two of my favorite examples are Nacos and Sentinel.

Nacos is a dynamic service registry that allows services to discover each other, much like Netflix's Eureka. Sentinel flips the normal approach to traffic shaping and reliability that you see in Netflix's Hystrix, providing a single piece of infrastructure that's able to handle routing and recovery for calls between services.

Alibaba also has a framework that builds upon Spring AI called Spring AI Alibaba, providing agentic systems and service access to models more commonly used in that community. Spring AI is, of course, a one-stop shop for all things AI – chat models, multimodal models, RAG, agents, and so on. It supports more than 20 well-known models, but, unfortunately, not *all* of them, since it's hard to test and integrate all models when they're only available in one region of the world. Additionally, Alibaba's integration extends Spring AI in new and interesting ways for new and interesting models.

Conclusion

We're at the end of our time together, brief though it was. We've done so much and come so far!

We've looked at the following:

- Setting up a proper working environment
- Modeling our data's domain
- Working with a SQL database
- HTTP clients
- HTTP
- REST
- HATEOAS
- GraphQL

- gRPC
- Authentication and authorization
- One-time tokens
- Passkeys
- OAuth and the Spring Authorization Server
- API gateways
- Scalability with Java 21's virtual threads
- Speed with GraalVM native images
- Packaging an application for production with Buildpacks and Docker

Next steps

Of course, the joyous journey to production is forever ahead of us. It's a pretty smooth ride, but there may, of course, be some bumps. I hope you'll consider the amazing resources out there to help you along the way:

- The official Spring home page (`https://spring.io/`)
- The official Spring guides on various topics (`https://spring.io/guides`)
- The Spring Initializr (`https://start.spring.io/`)
- The official Spring YouTube channel (`https://www.youtube.com/@springsourcedev`)
- The YouTube channel I run with some friends (`https://www.youtube.com/@coffeesoftware`)

Hopefully, you feel better prepared, you learned something, and you're ready to meet whatever challenges lie ahead.

You are well prepared. The web, for all its age and evolution, remains the most powerful, resilient, and universal platform we have. It's not just enduring; it's thriving. This journey through the "real" web reminds us that HTTP, humble though it may seem, continues to power everything from APIs to full-blown applications across the globe. It's not going anywhere, and the possibilities it offers developers are only expanding.

So let's lean in. The web's best days may still lie ahead. With the right tools (such as Spring) and the right mindset, we're poised to build faster, smarter, and more delightfully connected experiences than ever before. The web isn't just still alive; it's just getting warmed up.

Index